



REGRESSÃO LOGÍSTICA

APRENDIZAGEM DE MÁQUINA

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



O QUE É REGRESSÃO LOGÍSTICA?

- A Regressão Logística é, curiosamente, um algoritmo de classificação (e não de regressão, apesar do nome) muito utilizado em Machine Learning e Estatística. Ela é usada para prever a probabilidade de um evento pertencer a uma determinada classe.
- O caso mais comum é a classificação binária, onde o resultado pode ser apenas duas opções (ex: "Sim/Não", "Spam/Não Spam", "Doente/Saudável", que mapeamos matematicamente como 1 ou 0).

O PROBLEMA COM A REGRESSÃO LINEAR

- Na regressão linear, tentamos prever um valor contínuo através da equação da reta:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

- Ou, em notação vetorial (onde β são os pesos ou coeficientes e x são as variáveis de entrada – features):

$$z = \beta^T \cdot x$$

O PROBLEMA COM A REGRESSÃO LINEAR

- O problema de usar essa equação para classificação é que o resultado z pode assumir qualquer valor entre infinito positivo e infinito negativo. Como queremos uma probabilidade, precisamos de um resultado que esteja estritamente no intervalo entre 0 e 1.
- Para "espremer" esse valor de z para dentro do intervalo de 0 a 1, a regressão logística passa a saída linear por uma função matemática chamada Função Logística ou Função Sigmoid.

A FUNÇÃO SIGMOIDE

- A fórmula da função sigmoide é:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Se z for um número positivo muito grande, e^{-z} chega perto de 0, e a função resulta em um valor muito próximo de 1.
- Se z for um número negativo muito grande, e^{-z} tende ao infinito, e a função resulta em um valor muito próximo de 0.
- Se $z = 0$, a função resulta em exatos 0.5.

A EQUAÇÃO DA REGRESSÃO LOGÍSTICA

- Substituindo a nossa equação linear dentro da função sigmoide, obtemos a fórmula final que nos dá a probabilidade $\hat{y}$ do evento pertencer à classe 1 :

$$\hat{y} = P(y = 1 | x) = \frac{1}{1 + e^{-z}}$$

- Para tomar uma decisão final (classificar), estabelecemos um limiar de decisão (**decision boundary**). O mais comum é 0.5
 - Se $\hat{y} \geq 0.5 \rightarrow$ prevemos a classe 1.
 - Se $\hat{y} < 0.5 \rightarrow$ prevemos a classe 0.

A FUNÇÃO DE CUSTO (LOG LOSS / ENTROPIA CRUZADA)

- Para treinar o modelo, precisamos de uma forma de medir o quão erradas estão as nossas previsões para podermos corrigir os coeficientes (β).
- Na regressão linear, usamos o Erro Quadrático Médio (MSE).
- Na regressão logística, colocar a sigmoide no MSE gera uma curva cheia de vales (mínimos locais), o que dificulta o aprendizado do algoritmo.
- Por isso, usamos a função de Log Loss (ou Entropia Cruzada Binária).

A FUNÇÃO DE CUSTO (LOG LOSS / ENTROPIA CRUZADA)

- O custo para uma única previsão é definido como:

$$\text{custo}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \cdot \log(1 - \hat{y})$$

- Por que essa fórmula funciona?
 - Se a classe real for $y = 1$: O segundo termo $(1 - y)$ zera. A fórmula vira $\log(\hat{y})$. Se o modelo prever $\hat{y} \approx 1$ (acerto), o custo é 0. Se prever $\hat{y} \approx 0$ (erro grave), o custo vai para o infinito.
 - Se a classe real for $y = 0$ O primeiro termo y zera. A fórmula vira $-\log(1 - \hat{y})$. Aplica-se a mesma lógica invertida.

A FUNÇÃO DE CUSTO TOTAL

- A função de custo total $J(\beta)$ para todo o conjunto de dados com m exemplos é a média de todos os custos individuais:

$$J(\beta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \cdot \log(1 - \hat{y}^{(i)})]$$

OTIMIZAÇÃO (GRADIENTE DESCENDENTE)

- O objetivo do treinamento é encontrar os valores dos coeficientes β que minimizem essa função de custo $J(\beta)$.
- O método mais tradicional para fazer isso é o Gradiente Descendente.
- O algoritmo calcula a derivada (o gradiente) da função de custo em relação a cada peso β_j e atualiza os pesos passo a passo na direção oposta ao gradiente:

$$\beta_j = \beta_j - \alpha \frac{\partial J(\beta)}{\partial \beta_j}$$

OTIMIZAÇÃO (GRADIENTE DESCENDENTE)

- A derivada matemática simplificada resulta em uma regra de atualização incrivelmente elegante:

$$\beta_j = \beta_j - \alpha \frac{1}{m} \sum_{i=1}^m [\hat{y}^{(i)} - y^{(i)}] \cdot x_j^{(i)}$$

- Onde α é a taxa de aprendizado (**learning rate**), que controla o tamanho do passo que o modelo dá a cada iteração.

IMPLEMENTAÇÃO NO JUPYTER NOTEBOOK



OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: **Google Class Room**
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@fatecjahu.edu.br

